설비 고장을 예측하는 AI 데이터 분석

추세·주기·변동성 해석

추세와 변동성

posco

CONTENTS

목차

01 시계열 데이터와 추세

02 추세와 이동평균

03 변동성 분석

posco

posco

01

시계열 데이터와 추세

설비 센서값이 시간에 따라 어떻게 변하는지 데이터로 읽는 법

설비는 멈추기 전에 신호를 보낸다

고장의 신호는 데이터에 남고, 그 신호를 읽어 미리 대비하는 일

설비 고장은 갑자기 오지 않고 **천천히** 다가옴

작은 변화는 사람 눈에 안 보이지만 **데이터**에는 남음

이번 파트 목표는 그 신호를 읽어 **고장을 미리 대비**하는 일

posco

데이터를 읽으면 고장에 대비할 수 있다

센서가 기록한 변화를 직접 읽고 해석하는 일, 도구는 평균과 흩어진 정도면 충분

01 · 센서 측정

온도·압력·진동을 일정 간격으로 기록

값을 시간순으로 늘어놓으면 사람이 놓친 변화가 보임

02 · 변화 읽기

복잡한 모델 전에 눈으로 먼저 해석

데이터가 무엇을 말하는지 우리 눈으로 해석하는 힘 기르기

03 · 도메인 자산

설비를 이해하는 현장 지식이 무기

AI는 도구일 뿐, 가장 큰 자산은 현장 지식

예지정비 도입 기업의 비계획 정지 약 30~50% 감소 보고

시계열 데이터란 무엇인가

시간 순서대로 측정해 기록한 데이터 — 핵심은 순서

시계열 데이터는 시간 순서대로 측정해 기록한 데이터

핵심은 **순서**, 순서를 바꾸면 의미가 사라짐

매일 잰 몸무게, 주식 차트, 일별 기온, 설비 센서값이 모두 시계열

> **이웃한 값끼리 연결되어 흐름을 이룸 — 일반 데이터와의 결정적 차이**

posco

일반 데이터와 시계열 데이터의 차이

관심사가 다름 — 현재 값인가, 변화의 흐름인가

구분	일반	시계열
핵심	항목별 값	**시간순 변화**
순서	**바꿔도 됨**	**못 바꿈**
예시	학생별 키	일별 기온

시계열의 순서 — 일반 데이터처럼 섞으면 변화의 흐름이 사라짐

posco

시계열의 두 축 — 시간과 값

언제 측정했는가 × 그때 값이 얼마였는가의 두 짝

01 · 가로축 — 시간

왼쪽 → 오른쪽으로 시간이 흐름

설비에서는 가동 횟수나 측정 시각이 가로축 자리

cycle · 측정 시각 · 날짜

02 · 세로축 — 값

위로 갈수록 값이 큼

센서가 측정한 온도·압력·진동 같은 값이 세로축 자리

온도 · 압력 · 진동

선이 오른쪽으로 가며 올라가면 — 시간이 지날수록 값이 커진다는 뜻

그래프를 볼 때 반드시 확인할 두 가지

축의 단위와 범위를 먼저 확인해야 같은 데이터에 속지 않음

01

가로축 단위 확인

한 칸이 1초인가, 1시간인가, 한 번 가동인가에 따라 의미가 완전히 달라짐

02

세로축 눈금 확인

같은 데이터도 범위에 따라 평평하게 또는 뚜렷한 상승으로 보임

예 — 641→646℃를 0~1000℃로 그리면 평평, 640~647℃로 좁히면 뚜렷한 상승

C-MAPSS 데이터 구조 살펴보기

NASA 공개 항공 엔진 데이터 — 가동부터 고장까지 센서값의 시계열

NASA가 공개한 항공기 엔진 데이터, 가동부터 고장까지 센서값 기록

표 형태로 **한 줄이 한 시점의 기록**

같은 엔진 번호의 줄을 가동 주기 순서로 늘어놓으면 그 엔진의 시계열

> **정상에서 고장으로 가는 전 과정 — 고장 예측 학습의 표준 데이터셋**

posco

샘플 데이터 주요 열

엔진 번호 + 가동 주기 + 운영 설정 3 + 센서 5 — 학습용 단순 구성

열 이름	의미	예시
`unit_nr`	엔진 번호	
`time_cycles`	가동 주기	`1, 2`
`s_2`	온도 센서	`641.27`

엔진 5대 · 수명 다양 (155~287주기) · 실제 C-MAPSS는 센서 21개지만 학습엔 5개로 충분

cycle이 시간축이 되는 이유

시계열의 핵심은 순서, 날짜가 없어도 번호만 분명하면 충분

C-MAPSS에는 날짜가 없고 가동 주기 번호(1, 2, 3)만 존재

시계열의 핵심은 **순서**, 번호 순서만 분명하면 훌륭한 시간축

가동 주기가 커질수록 시간이 흐른 것, 엔진은 더 오래 사용된 것

> **cycle 기반과 날짜 기반의 차이 — 가로축에 뭘 쓰느냐일 뿐, 분석법은 동일**

cycle 기반과 날짜 기반의 차이

설비 특성에 따라 가로축의 단위가 달라질 뿐, 분석 방법은 같음

01 · cycle 기반

가동 횟수로 기록

한 번 가동이 의미 단위인 설비 — 항공 엔진, 프레스

1회차 → 2회차 → 3회차

02 · 날짜 기반

실제 시각으로 기록

24시간 돌아가는 공정 설비 — 하루 평균 같은 시간 분석 가능

1월 5일 → 8일 → 12일

이번 강의는 두 방식을 모두 다룸 — 어떤 데이터를 만나도 당황하지 않게

데이터 불러오기 3단계

파일 읽기 → 한 엔진만 골라내기 → 분석할 센서 고르기

STEP 1 · 불러오기

read_csv로
DataFrame 읽기

이미 배운 read_csv 그대로 사용

```
pd.read_csv(...)
```

STEP 2 · 엔진 선택

조건 필터링으로
한 엔진만 골라내기

섞으면 시간 순서가 엉김 — 한 번에 한 엔진만

```
df[df['unit_nr']==1]
```

STEP 3 · 센서 선택

분석할 센서 하나
열 이름으로 선택

s_2는 오래될수록 값이 올라 추세 학습에 좋음

```
engine1['s_2']
```

엔진을 안 골라내면 1번 마지막 주기 다음에 2번 첫 주기가 붙어 시간 흐름 엉킴 — 반드시 한 엔진씩

posco

데이터 불러오기와 엔진·센서 선택 코드

read_csv로 불러와 1번 엔진의 s_2 센서 선택

CODE

```
import pandas as pd
df = pd.read_csv('data/cmapss_fd001_sample.csv')
print(df.shape) # (1031, 10)
engine1 = df[df['unit_nr'] == 1]
print(engine1.shape) # (192, 10)
s2 = engine1['s_2']
```

posco

불러온 직후 반드시 확인할 3가지

head · shape · info — 분석 전 점검 습관

01 · HEAD()

처음 몇 줄로 데이터 모양 확인

값의 단위와 형식이 예상과 같은지 한눈에 보기

02 · SHAPE

행과 열의 개수 확인

의도한 대로 걸러졌는지 알 수 있는 지표

03 · INFO()

각 열의 자료형 점검

숫자가 문자로 읽혔는지 확인 — float64면 정상

숫자처럼 보여도 문자로 저장되면 평균·표준편차 계산 불가

posco

실습 1. 센서 시계열 그려보기

한 엔진의 센서 값을 회차 순서대로 선 그래프로 보기

목표

한 엔진의 센서 값을 회차 순서대로 선 그래프로 그려 변화 방향 읽기

단계

· 2번 엔진을 골라 회차를 가로축으로 준비

· 센서 하나를 선 그래프로 그리기

· 제목·축 이름을 붙이고 값이 오르내리는 방향 읽기

예상 결과

회차가 늘수록 값이 서서히 오르는 곡선

posco

실습 1. 센서 시계열 그려보기 정답

한 엔진의 센서 값을 회차 순서대로 선 그래프로 보기

CODE · 실습 1

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('cmapss_fd001_sample.csv')
e2 = df[df['unit_nr'] == 2].reset_index(drop=True)
plt.plot(e2['time_cycles'], e2['s_4'])
plt.title('Engine 2 - s_4'); plt.xlabel('cycle'); plt.ylabel('s_4')
plt.show()
```

02

추세와 이동평균

노이즈에 가려진 진짜 변화를 이동평균으로 드러내는 법

posco

그래프가 들쭉날쭉한 이유 — 노이즈

센서값이 작게 위아래로 떨리는 자연스러운 현상, 없앨 게 아닌 함께할 대상

센서값이 작게 위아래로 떨리는 현상을 **노이즈**(잡음)라고 함

측정 오차, 순간 조건 변화, 외부 요인이 합쳐진 결과

노이즈 없는 완벽히 매끈한 센서 데이터는 현실에 없음

> **목표는 노이즈를 없애는 일이 아니라 노이즈에 가려진 진짜 변화를 찾는 일**

posco

추세란 무엇인가

자잘한 떨림을 무시하고 길게 봤을 때 데이터의 전체 방향

01 · 상승



값이 우상향

s_2 — 가동 주기가 늘수록 온도 상승

02 · 하강



값이 우하향

s_7 — 시간이 지날수록 값 하강

03 · 평탄



수평 유지

안정 상태 — 큰 방향 변화 없음

한 점 한 점에 집착하지 말고 **눈을 가늘게 뜨고 멀리서 보기** — 설비의 장기 건강 상태가 추세

posco

노이즈 때문에 추세가 안 보이는 문제

누가 봐도 같은 결론이 나오는 객관적 계산이 필요한 이유

완만한 추세는 노이즈에 가려 눈으로 읽기 어려움

같은 그래프를 보고도 **사람마다 다르게** 판단

누가 봐도 같은 결론이 나오는 객관적 계산이 필요

> **"그래프 보고 알아서 판단"은 분석이 아님 — 같은 결론을 만드는 계산이 분석**

posco

이동평균 개념

구간 평균을 한 칸씩 미끄러뜨리며 반복 계산 — 코로나 7일 이동평균과 같은 원리

일정 구간으로 묶어 평균을 내되 그 구간을 한 칸씩 **이동**시키는 방법

윈도우(window)는 한 번에 묶는 값의 개수

작은 창문을 데이터 위에서 미끄러뜨리며 평균을 반복 계산

EXAMPLE — window 3

`[10, 12, 15, 14, 18] → [12.33, 13.67, 15.67]`

손바닥으로 가리고 창문만큼만 보며 천천히 오른쪽으로 미는 모습을 상상

posco

이동평균이 추세를 드러내는 원리

노이즈는 상쇄되어 사라지고 추세는 살아남음 — 이것이 평활화(smoothing)

01 · 상쇄

노이즈는 위아래로 튀어
평균 속에서 상쇄

`100, 102, 98 → avg 100`

+2와 −2가 만나 0이 됨

02 · 잔존

한 방향 추세는
평균을 내도 남음

`avg 100.5 → 102.5`

추세는 평균으로도 살아남음

03 · 평활화

데이터를 부드럽게
다듬는 일

smoothing

주식 5일선, 코로나 7일 평균과 같은 원리

한 문장 — 노이즈는 상쇄되어 사라지고, 추세는 살아남음

이동평균 구하기 — rolling().mean()

rolling으로 윈도우 정하고 mean 붙이기 — 앞부분 NaN은 정상

CODE

```
import pandas as pd
s = pd.Series([10, 12, 15, 14, 18])
print(s.rolling(window=3).mean().tolist())
# [nan, nan, 12.33, 13.67, 15.67]
# 앞 2개는 묶을 값 부족 -> NaN
```

실습 2. 이동평균선으로 추세 보기

떨리는 원본 위에 매끈한 이동평균선을 겹쳐 추세 보기

목표

떨리는 원본 위에 매끈한 이동평균선을 겹쳐 추세를 드러내기

단계

· 창 크기 10으로 이동평균을 구하기

· 원본과 이동평균을 한 그래프에 겹쳐 그리기

· 범례를 붙이고 이동평균이 추세를 드러냄을 확인

예상 결과

들쭉날쭉한 원본 사이로 매끈한 상승 추세선

posco

실습 2. 이동평균선으로 추세 보기 정답

떨리는 원본 위에 매끈한 이동평균선을 겹쳐 추세 보기

CODE · 실습 2

```
ma10 = e2['s_4'].rolling(window=10).mean()
plt.plot(e2['time_cycles'], e2['s_4'], alpha=0.5, label='raw')
plt.plot(e2['time_cycles'], ma10, label='MA(10)')
plt.title('Engine 2 s_4 - raw vs MA'); plt.legend()
plt.show()
```

윈도우 크기의 의미

분석 목적에 맞는 안경 도수를 고르는 것과 같음

윈도우 크기는 한 번에 묶어 평균 낼 값의 **개수**

클수록 더 매끈하지만 변화에 둔감, 작을수록 거칠지만 민감

분석 목적에 맞는 안경 도수를 고르는 것과 같음

192주기 데이터 권장

적정 윈도우 10 ~ 30 — 정답은 없고 목적에 맞게 고르기

높은 도수 안경 = 흐릿하지만 큰 윤곽이 보임 · 낮은 도수 = 선명하지만 어수선

윈도우가 작을 때와 클 때의 차이

매끈함과 반응 속도의 맞교환 — 무엇을 보고 싶은지가 기준

구분	작은 윈도우	큰 윈도우
선 모양	거칠게 떨림	매끈함
변화 반응	빠름	느림
빈 앞부분	적음	많음

큰 윈도우가 무조건 좋은 게 아님 — 매끈한 대신 둔감하고 앞부분이 많이 빔

실습 3. 윈도우 크기 바꿔 비교하기

창 크기를 바꿔 매끈함과 빈 구간의 맞교환 관찰

목표

창 크기를 5·20·50으로 바꿔 매끈함과 빈 구간의 맞교환을 관찰

단계

· 원본을 옅게 그린 뒤 세 가지 창 크기 이동평균을 겹치기

· 창이 클수록 더 매끄럽지만 앞쪽 빈 구간이 길을 확인

· 범례로 어느 선이 어느 창인지 구분

예상 결과

창이 클수록 매끈하지만 시작 부분 빈 구간이 늘어남

posco

실습 3. 윈도우 크기 바꿔 비교하기 정답

창 크기를 바꿔 매끈함과 빈 구간의 맞교환 관찰

CODE · 실습 3

```
plt.plot(e2['time_cycles'], e2['s_4'], alpha=0.3, label='raw')
for w in [5, 20, 50]:
    plt.plot(e2['time_cycles'], e2['s_4'].rolling(w).mean(), label=f'MA({w})')
plt.title('window size comparison'); plt.legend()
plt.show()
```

추세 방향 판단 — 상승·하강·평탄

데이터는 사실까지, 해석은 센서 의미를 아는 사람 몫

이동평균선의 방향으로 상승·하강·평탄을 객관적으로 확인

방향만으로는 부족, 그 센서의 **의미**(정상 방향)와 함께 해석

정상 방향과 다르거나 변화가 너무 빠르면 주의 신호

데이터 vs 해석

"온도가 오른다"는 사실, "온도가 올라 위험하다"는 해석

같은 상승도 센서에 따라 의미가 정반대 — 변화 속도도 함께 봐야 함

실습 4. 여러 센서 추세 방향 비교

여러 센서의 이동평균을 함께 그려 방향 비교

목표

여러 센서의 이동평균을 함께 그려 상승·하강 방향을 비교

단계

· 대상 센서마다 창 크기 20 이동평균을 구하기

· 한 그래프에 겹쳐 그리고 범례를 붙이기

· 각 센서가 오르는지 내리는지 방향 판정

예상 결과

어떤 센서는 상승, 어떤 센서는 하강하는 서로 다른 방향

posco

실습 4. 여러 센서 추세 방향 비교 정답

여러 센서의 이동평균을 함께 그려 방향 비교

CODE · 실습 4

```
for col in ['s_4', 's_7', 's_11']:
    plt.plot(e2['time_cycles'], e2[col].rolling(20).mean(), label=col)
plt.title('trend comparison (MA20)'); plt.legend()
plt.show()
# s_4·s_11 상승,  s_7 하강
```

03

변동성 분석

얼마나 불안한가 — 평균보다 먼저 나타나는 조기 경고

추세만으로 부족한 이유

잔잔한 호수와 파도치는 바다 — 평균 수위가 같아도 완전히 다른 상태

추세는 **평균의 방향**만 알려줌, 값의 불안정한 정도는 못 알려줌

평균이 같아도 매끈하게 오르는 설비와 심하게 출렁이며 오르는 설비는 다름

고장은 종종 평균보다 **변동성**으로 먼저 신호를 보냄

심하게 출렁이며 오르는 설비가 훨씬 위험 — 흔들림으로 먼저 나타나는 고장

변동성이란 무엇인가

값이 평균을 중심으로 얼마나 크게 흔들리는지

변동성은 값이 평균을 중심으로 **얼마나 크게 흔들리는지**

작으면 안정적, 크면 평균에서 멀리 위아래로 튐

추세가 "어디로 가는가"라면, 변동성은 "얼마나 불안한가"

변동성 작음

매일 같은 시간에 일어나는 사람 — 안정적

변동성 큼

새벽 5시·낮 12시를 오가는 사람 — 불안정

값이 오르든 내리든 상관없이 평균에서 얼마나 멀리 튀느냐만 봄

posco

표준편차 — 흩어진 정도 복습

표준편차 = 흩어진 정도 = 변동성을 재는 자

표준편차는 값들이 평균에서 평균적으로 얼마나 떨어졌는지 나타내는 수

작으면 평균 근처에 모임, 크면 멀리 흩어짐

`[10, 10, 10, 10]`

std = 0

흩어짐 없음

`[8, 12, 9, 11]`

std ≈ 1.7

조금 흩어짐

`[2, 18, 5, 15]`

std ≈ 6.9

멀리 흩어짐

단위가 원래 값과 같음 — "std 2도"는 "평균에서 2도쯤 흔들림"으로 직관 해석

posco

이동표준편차 개념

이동평균과 한 쌍 — 마지막에 평균 대신 표준편차를 낼 뿐

윈도우를 이동하며 구간마다 **표준편차**를 계산

변동성이 시간에 따라 어떻게 변하는지 추적

이동평균과 한 쌍, 마지막에 평균 대신 표준편차를 낼 뿐

이동평균

```
.rolling(20).mean()
```

이동표준편차

```
.rolling(20).std()
```

하나를 알면 하나는 공짜 — 윈도우 미끄러뜨리는 방식까지 동일

posco

변동성 증가가 설비에 갖는 의미

변동성은 평균 변화보다 먼저 나타나는 조기 경고

01 · 정상 설비

값이 일정하게
변동성이 작음

정해진 범위에서 안정적으로 움직임

02 · 문제 시작

값이 불규칙해져
변동성이 커짐

베어링 마모로 진동이 들쭉날쭉, 부품 헐거움으로 압력 불안정

03 · 조기 경고

평균보다 먼저
나타나는 신호

나사 헐거워진 차 — 평균 진동은 그대로여도 덜컹임이 커짐

평균이 아직 안 변했을 때도 변동성은 먼저 움직임 — 변동성을 따로 보는 이유

posco

이동표준편차 구하기 — mean 대신 std

이동평균과 거의 같음 — mean을 std로 바꾸기만

CODE

```
std20 = engine1['s_3'].rolling(20).std()
# 초반 구간 0.931, 후반 구간 1.862
# 후반 변동성 약 2배 증가
```

실습 5. 이동표준편차로 변동성 구간 찾기

원본과 이동표준편차를 위아래로 나눠 그려 구간 찾기

목표

원본과 이동표준편차를 위아래로 나눠 그려 흔들림이 큰 구간을 찾기

단계

· 창 크기 20으로 이동표준편차를 구하기

· 위칸에 원본, 아래칸에 이동표준편차를 그리기

· 표준편차가 치솟는 구간을 변동성이 큰 구간으로 표시

예상 결과

후반부에서 이동표준편차가 커지는 불안정 구간

posco

실습 5. 이동표준편차로 변동성 구간 찾기 정답

원본과 이동표준편차를 위아래로 나눠 그려 구간 찾기

CODE · 실습 5

```
std20 = e2['s_3'].rolling(window=20).std()
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(11, 6), sharex=True)
ax1.plot(e2['time_cycles'], e2['s_3']); ax1.set_title('s_3 raw')
ax2.plot(e2['time_cycles'], std20, color='red')
ax2.set_title('s_3 moving std(20)')
plt.tight_layout(); plt.show()
# std20 후반(idx180) ≈ 1.053 (커짐)
```

posco

평균±표준편차 밴드 개념

가운데 선의 방향 = 추세, 띠의 폭 = 변동성

이동평균선 위아래로 표준편차만큼 띠를 둘러 그린 것

가운데 선은 **추세**, 띠의 폭은 **변동성**을 보여줌

띠가 넓어지는 구간은 불안정해지는 구간

구성 공식

$$MA \pm 2 \times STD$$ — 가운데 선 + 위아래 띠

표준편차 2배 밴드 — 대부분 정상값이 안에 들어와, 크게 벗어난 점은 이상값

실습 6. 평균과 변동성 밴드 함께 그리기

이동평균선에 표준편차 두 배 띠를 둘러 정상 범위 보기

목표

이동평균선에 표준편차 두 배 띠를 둘러 정상 범위를 시각화

단계

· 창 크기 20으로 이동평균과 이동표준편차를 구하기

· 원본과 이동평균을 그리기

· 평균에 표준편차 두 배를 더하고 뺀 띠를 색으로 채우기

예상 결과

이동평균 주위로 폭이 변하는 정상 범위 띠

posco

실습 6. 평균과 변동성 밴드 함께 그리기 정답

이동평균선에 표준편차 두 배 띠를 둘러 정상 범위 보기

CODE · 실습 6

```
ma = e2['s_3'].rolling(20).mean(); sd = e2['s_3'].rolling(20).std()
plt.plot(e2['time_cycles'], e2['s_3'], alpha=0.4, label='raw')
plt.plot(e2['time_cycles'], ma, label='MA(20)')
plt.fill_between(e2['time_cycles'], ma - 2*sd, ma + 2*sd, alpha=0.15, label='band')
plt.legend(); plt.show()
# idx100 평균 1592.37, 표준편차 1.1
```

변동성 패턴 해석 — 점증과 급증

천천히 커지는지 한순간에 튀는지에 따라 다르게 읽기

01 · 점증



변동성이 서서히 우상향

점진적 마모·노화로 천천히 불안정 — **정비 계획 미리** 세우기

s_3가 이 예시

02 · 급증



어느 지점에서 가파른 솟구침

부품 손상·외부 충격 가능성 — **원인 즉시** 확인

원본·운영 기록 동시 확인 필수

급증이라고 항상 고장은 아님 — 원본과 운영 기록을 함께 보고 원인 판단

posco

실습 7. 변동성 패턴 해석

여러 센서의 이동표준편차로 점증·급증 패턴 비교

목표

여러 센서의 이동표준편차를 나눠 그려 점증·급증 패턴을 비교

단계

· 대상 센서마다 창 크기 20 이동표준편차를 구하기

· 센서별로 칸을 나눠 각각 그리기

· 서서히 커지는 점증과 갑자기 튀는 급증을 구분

예상 결과

센서마다 다른 변동성 패턴 — 완만한 증가와 급한 튐

posco

실습 7. 변동성 패턴 해석 정답

여러 센서의 이동표준편차로 점증·급증 패턴 비교

CODE · 실습 7

```
sensors = ['s_2', 's_3', 's_11']
fig, axes = plt.subplots(3, 1, figsize=(11, 8), sharex=True)
for ax, col in zip(axes, sensors):
    ax.plot(e2['time_cycles'], e2[col].rolling(20).std())
    ax.set_title(col + ' moving std(20)')
plt.tight_layout(); plt.show()
```